Una **relación** UML puede tener simultáneamente:

- un tipo: asociación, agregación, composición o generalización;
- una navegabilidad: unidireccional o bidireccional;
- una multiplicidad: `1`, `0..1`, `0..*`, etc.;
- nombres de roles: `conductor`, `vehículo`, `sensores`, etc.

1. Mapa general de los conceptos

Pregunta que quiero responder	Conceptos involucrados
¿Una clase sólo usa temporalmente a la otra?	Dependencia
¿Los objetos mantienen un vínculo?	Asociación
¿Quién conoce a quién?	Navegabilidad unidireccional o bidireccional
¿Una entidad forma parte de otra?	Agregación o composición
¿La parte puede existir independientemente?	Diferencia entre agregación y composición
¿Cuántos objetos pueden relacionarse?	Multiplicidad
¿Un concepto es un tipo particular de otro?	Generalización, especialización y herencia
¿La relación tiene datos propios?	Clase de asociación

---

2. Clase, objeto, relación y vínculo

Estos conceptos distinguen el nivel general del caso concreto.

Concepto	Qué representa	Ejemplo
Clase	Una definición general o tipo	`Persona`, `Taxi`
Objeto	Una instancia concreta de una clase	`pedro`, `taxi25`
Relación	Posibles conexiones entre objetos de dos clases	Una `Persona` conduce un `Taxi`
Vínculo o enlace	La conexión entre dos objetos concretos	`pedro` conduce `taxi25`

---

3. Dependencia, asociación unidireccional y asociación bidireccional

Concepto	Significado	¿Se conserva el vínculo?	UML	Implementación típica en el programa
Dependencia	Una clase usa **temporalmente** a otra	No	Flecha discontinua	Parámetro o variable local
Asociación unidireccional	***A*** conoce **permanentemente** a ***B***	Sí, desde `A`	Línea continua con flecha hacia `B`	`B*` como atributo de `A`
Asociación bidireccional	`A` conoce a `B` y `B` conoce a `A`	Sí, en ambos sentidos	Línea continua sin flecha	Ambas clases guardan referencias
Asociación sin navegabilidad especificada	Se sabe que existe el vínculo, pero no se decide quién podrá acceder a quién	No está decidido	Línea continua sin flechas	Se define posteriormente

Dependencia: uso temporal

Una impresora recibe un documento para imprimirlo, pero no lo guarda como parte de su estado.

```
class Documento
{
};

class Impresora
{
public:
    void imprimir(const Documento& documento)
    {
        // Usa documento solamente durante esta llamada
    }
};
```

La relación existe mientras se ejecuta:

```
impresora.imprimir(documento);
```

Al terminar `imprimir()`, la impresora no conserva necesariamente ninguna conexión con ese documento.

Asociación unidireccional

Una persona conoce el taxi que conduce, pero el taxi no conoce a la persona.

```
class Taxi
{
};

class Persona
{
private:
    Taxi* taxi = nullptr;  // No es propietaria del taxi

public:
    void asignarTaxi(Taxi* nuevoTaxi)
    {
        taxi = nuevoTaxi;
    }
};
```

La navegación permitida es:

```
Persona → Taxi
```

Desde `Persona` podemos obtener el taxi. Desde `Taxi` no podemos obtener la persona.

Asociación bidireccional

Ambos objetos se conocen.

```
class Taxi;

class Persona
{
private:
    Taxi* taxi = nullptr;

public:
    void asignarTaxi(Taxi* nuevoTaxi)
    {
        taxi = nuevoTaxi;
    }
};

class Taxi
{
private:
    Persona* conductor = nullptr;

public:
```

```
    void asignarConductor(Persona* nuevaPersona)
    {
        conductor = nuevaPersona;
    }
};
```

La navegación es:

```
Persona ↔ Taxi
```

Su desventaja es que hay que mantener ambos extremos consistentes:

```
persona.asignarTaxi(&taxi);
taxi.asignarConductor(&persona);
```

Si sólo se actualiza uno, los objetos quedan con información contradictoria.

Diferencia esencial

Si ocurre esto…	Relación probable
`A` recibe un `B` como parámetro y lo usa	Dependencia
`A` guarda una referencia hacia `B`	Asociación unidireccional
`A` guarda una referencia a `B` y `B` otra hacia `A`	Asociación bidireccional

---

4. Asociación, agregación y composición

Las tres representan vínculos estructurales, pero no tienen la misma semántica.

Característica	Asociación	Agregación	Composición
Idea principal	“Se relaciona con”	“Tiene una parte”	“Está compuesto por”
Relación todo-parte	No necesariamente	Sí, débil	Sí, fuerte
Vida de la parte	Independiente	Independiente	Depende del todo
Símbolo UML	Línea continua	Rombo blanco	Rombo negro
Posición del rombo	No hay rombo	Junto al todo	Junto al todo
C++ habitual	Puntero o referencia no propietaria	Puntero a objeto externo	Objeto por valor o `unique_ptr`
Ejemplo	Persona–Taxi	Ordenador–Pantalla	Empresa–Departamento

Asociación común

Una persona conduce un taxi. Ninguno es una parte física o lógica del otro.

```
class Taxi
{
};

class Persona
{
private:
    Taxi* taxiConducido = nullptr;
};
```

Esto no significa que la persona sea propietaria del taxi.

Agregación

Un ordenador **utiliza una pantalla**, pero la pantalla puede existir antes y después del ordenador.

```
class Pantalla
{
};

class Ordenador
{
private:
    Pantalla* pantalla;  // Objeto externo, no propietario

public:
    Ordenador(Pantalla* unaPantalla) : pantalla(unaPantalla)
    {
    }
};
```

Aplicación:

```
int main()
{
    Pantalla pantalla1;

    {
        Ordenador ordenador1(&pantalla1);
    }

    // Se destruye ordenador
    // pantalla continúa existiendo
}
```

La pantalla podría desconectarse y utilizarse en otro ordenador. Por eso es una relación todo-parte débil.

Composición

Una cinta está compuesta por su banda y sus cilindros. Esas partes pertenecen al objeto `CintaTransportadora`.

```cpp
#include <vector>

class Banda
{
};

class Cilindro
{
};

class CintaTransportadora
{
private:
    Banda banda;
    std::vector<Cilindro> cilindros;
};
```

Cuando se crea una cinta, se crean sus partes. Cuando se destruye la cinta, C++ destruye automáticamente:

- la `banda`;
- todos los objetos contenidos en `cilindros`.

También puede existir composición con memoria dinámica.

La pregunta decisiva

No preguntes solamente "¿una clase tiene a la otra?". Preguntá: Si destruyo el objeto completo, **¿la parte debería continuar existiendo según el modelo?**

- Si puede seguir existiendo → agregación.
- Si debe desaparecer con el todo → composición.
- Si ni siquiera representa una parte → asociación común.

Advertencia

Un puntero no significa automáticamente agregación.

El código ayuda a implementar la relación, pero la decisión principal es **semántica**: quién es propietario y quién controla el ciclo de vida.

---

5. Multiplicidad

La multiplicidad responde solamente: **¿Cuántos objetos de esta clase pueden estar relacionados con un objeto del otro extremo?**

Multiplicidad	Significado	Implementación posible en C++
`1`	Exactamente uno	Objeto, referencia obligatoria o puntero validado
`0..1`	Ninguno o uno	Puntero que puede ser `nullptr`
`*`	Cualquier cantidad	`std::vector`
`0..*`	Cero o más	`std::vector` posiblemente vacío
`1..*`	Uno o más	`std::vector` con validación de no vacío
`2..6`	Entre dos y seis	Colección con controles al agregar y eliminar
`3`	Exactamente tres	`std::array<T, 3>`

Cómo se lee

Supongamos:

```
Empresa 1 ◆──────────────── 1..* Departamento
```

La multiplicidad escrita cerca de `Departamento` indica cuántos departamentos puede tener una empresa: **una** empresa debe tener **uno o más** departamentos.

La multiplicidad se lee mirando desde el extremo contrario.

Ejemplo en C++

```cpp
#include <stdexcept>
#include <vector>

class Departamento
{
};

class Empresa
{
private:
    std::vector<Departamento> departamentos;

public:
    Empresa(const Departamento& inicial)
    {
        departamentos.push_back(inicial);
```

```
    }

    void agregarDepartamento(const Departamento& departamento)
    {
        departamentos.push_back(departamento);
    }
};
```

El constructor obliga a crear la empresa con al menos un departamento, implementando `1..*`. Un `vector` por sí solo no garantiza `1..*`, porque podría estar vacío. La restricción debe implementarse mediante constructores y métodos.

---

6. Multiplicidad, navegabilidad, rol y propiedad

Estas características pueden aparecer juntas, pero responden preguntas distintas.

Característica	Pregunta que responde	Ejemplo
Tipo de relación	¿Qué significado tiene el vínculo?	Composición
Multiplicidad	¿Cuántos objetos participan?	`1..*` departamentos
Navegabilidad	¿Quién puede acceder a quién?	`Empresa` → `Departamento`
Rol	¿Qué función cumple el objeto en la relación?	`departamentos`
Propiedad	¿Quién controla el ciclo de vida?	`Empresa` posee departamentos

Por ejemplo:

```
class Empresa
{
private:
    std::vector<Departamento> departamentos;
};
```

Podríamos interpretar:

- tipo: composición;
- multiplicidad de `Departamento`: `0..*`;
- navegabilidad: `Empresa` → `Departamento`;
- rol: `departamentos`;
- propiedad: `Empresa` controla su vida.

No son cinco relaciones diferentes. Son cinco características de una misma relación.

---

7. Generalización, especialización y herencia

Concepto	Punto de partida	Proceso	Resultado
Generalización	Varias clases particulares	Extraer características comunes	Una clase más general
Especialización	Una clase general	Agregar características particulares	Clases más específicas
Herencia	Clases ya relacionadas jerárquicamente	Implementar la relación en un lenguaje	Clase base y clases derivadas
Polimorfismo	Una jerarquía de herencia	Tratar objetos específicos mediante el tipo general	Código común para distintos subtipos

Generalización: de lo particular a lo general

Es la acción de crear una clase más general a partir de varias particulares. Observamos:

```
SensorTemperatura
SensorPresion
SensorNivel
```

Todas estas clases tienen características en común como identificador, estado, unidad, etc. Generalizamos esas características en una clase:

```
Sensor
```

El razonamiento va desde varias clases específicas hacia una clase general.

Especialización: de lo general a lo particular

Es la acción de generar clases particulares a partir de una general. Partimos de:

```
Sensor
```

y definimos tipos particulares:

```
SensorTemperatura
SensorPresion
SensorNivel
```

Cada clase especializada agrega características o comportamientos propios.

Herencia: implementación en C++

```cpp
class Sensor
{
public:
    virtual float medir() const = 0;
    virtual ~Sensor() = default;
};

class SensorTemperatura : public Sensor
{
public:
    float medir() const override
    {
        return 25.0f;
    }
};

class SensorPresion : public Sensor
{
public:
    float medir() const override
    {
        return 1.2f;
    }
};
```

En UML, el triángulo blanco apunta hacia la clase más general:

```
SensorTemperatura ————▷ Sensor
SensorPresion     ————▷ Sensor
```

En C++:

- `Sensor` es la clase base;
- `SensorTemperatura` es una clase derivada;
- `SensorPresion` es otra clase derivada.

La diferencia más importante

Generalización y especialización son dos formas de recorrer la misma jerarquía:

```
SensorTemperatura ——▷ Sensor ◁—— SensorPresion
       específico       general       específico
```

- Subir hacia `Sensor`: generalización.
- Bajar hacia `SensorTemperatura`: especialización.
- Escribir `class SensorTemperatura : public Sensor`: herencia en C++.

No son tres relaciones UML diferentes.

---

8. Herencia frente a agregación y composición

Estas relaciones también suelen confundirse porque todas organizan clases jerárquicamente.

Característica	Herencia	Agregación/composición
Pregunta	¿Es un tipo de?	¿Es parte de o tiene un?
Relación	General–específico	Todo–parte
Palabra útil	“Es un”	“Tiene un”
Ejemplo	Un motor CC es un motor	Una cinta tiene un motor
UML	Triángulo blanco	Rombo blanco o negro
C++	`: public ClaseBase`	Atributo objeto, puntero o colección

Herencia

```cpp
class Motor
{
public:
    void arrancar()
    {
    }
};

class MotorCC : public Motor
{
};
```

Podemos afirmar: Un `MotorCC` es un `Motor`.

Composición

```cpp
class CintaTransportadora
{
private:
    MotorCC motor;
};
```

Podemos afirmar: Una `CintaTransportadora` tiene un `MotorCC`.

---

9. Clase de asociación

Una clase de asociación se utiliza cuando la propia relación necesita almacenar información. Supongamos:

```
Persona conduce Taxi
```

Una asociación común alcanza si sólo necesitamos conocer qué persona conduce qué taxi. Pero pueden aparecer datos como:

- fecha de inicio;
- turno;
- horas conducidas;
- licencia utilizada.

Esos datos no pertenecen exclusivamente a `Persona` ni a `Taxi`: pertenecen a la relación `Conduccion`.

Asociación común	Clase de asociación
Sólo representa un vínculo	El vínculo tiene datos propios
`Persona*` dentro de `Taxi`	Se crea una clase `Conduccion`
Adecuada para relaciones simples	Adecuada cuando la relación necesita estado o comportamiento

En C++:

```cpp
#include <string>

class Persona
{
};

class Taxi
{
};

class Conduccion
{
private:
    Persona* conductor;
    Taxi* taxi;
    std::string turno;
    int horasConducidas;
```

```
public:
   Conduccion(Persona* unaPersona, Taxi* unTaxi, const std::string& unTurno)
      : conductor(unaPersona), taxi(unTaxi), turno(unTurno),horasConducidas(0)
    {
    }

    void registrarHoras(int horas)
    {
        horasConducidas += horas;
    }
};
```

`Conduccion` representa un vínculo concreto y almacena la información correspondiente a ese vínculo.

---

10. Tabla final para decidir rápidamente

Situación encontrada en la consigna	Relación probable
`A` usa un `B` solamente para ejecutar un método	Dependencia
`A` conserva una referencia hacia `B`	Asociación
Sólo `A` necesita encontrar a `B`	Asociación unidireccional
`A` debe encontrar a `B` y `B` debe encontrar a `A`	Asociación bidireccional
`B` es parte de `A`, pero puede existir independientemente	Agregación
`B` pertenece exclusivamente a `A` y desaparece con él	Composición
`B` es un tipo particular de `A`	Generalización/herencia
La relación posee datos propios	Clase de asociación
Necesito indicar cuántos objetos participan	Multiplicidad
Necesito indicar quién conoce a quién	Navegabilidad

La idea central es:

```
ES UN       → herencia
USA A       → dependencia
CONOCE A    → asociación
TIENE UN    → agregación o composición
CUÁNTOS     → multiplicidad
QUIÉN CONOCE A QUIÉN → navegabilidad
```